

NAME: i

STUDENT ID:

Remember to also initial in the upper right-hand header of each page.

Due Friday Oct. 9th at the beginning of class.

Portfolio 4: Indefinite Iteration

To succeed in these problems you will need to read and engage with the embedded activities in the **Runestone textbook chapter** (worth 0.5 of a grade percentage point):

- 6: Iteration

Note that these activities are focused on this week's theme, but they are designed to utilize skills from all your previous portfolios as well. You should use your judgement to decide which tasks are best solved with which skills you have accumulated so far.

For completing and self-correcting 70% of the below activities, you will earn 1 grade percentage point. Additionally, for participating in a share-out of 1 of the activities, you will earn 0.5 of a grade percentage point.

Note that some activities we will plan to start in class and others you need to start on your own. You should finish each activity before its in-class share-out in order to correct it before final submission.

Remember to print and attach your programs for the coding activities. Programs should be submitted to gradescope for autograder results, but credit is giving to programs printed and included in this portfolio, on completion, regardless of autograder results. Programs should be marked up by hand for corrections.

Table of Contents

This portfolio includes the following 10 activities and the report of your weekly share-out:

Part	Problem/Task	Page	In-class Start	In-class Share-out	Submission Format
A1	Written prompt	2	F 10/2	F 10/2	written
A2	Python (largest number, congress)	3	M 10/5	M 10/5	GS, print
A3	Written questions (a bit of review)	5	oyo	M 10/5	written
A4	Python (leap year, dice)	7	oyo	W 10/7	GS, print
A5	Written questions	9	oyo	W 10/7	written
A6	Python (cash register, sum of powers)	11	W 10/7	W 10/7	GS, print
A7	Reeborg (around the world, climbing)	13	Th 10/8	Th 10/8	GS, print
A8	Tracing while loops	15	Th 10/8	Th 10/8	written
A9	Python (primes)	17	oyo	Th 10/8	GS, print
A10	Python (number guessing)	19	oyo	Th 10/8	GS, print
B1	Share-out report	21	oyo		written
B2	Share-out report (optional)	22	oyo		written

A1) Prompt: What elements (if any) of decision making require or constitute thinking?

A2) Largest Number & Running for Congress

1. Largest number: Write a function that uses a while loop to ask the user to enter many numbers, then returns the largest number entered. The prompt asking for a number should indicate that entering -1 will cause the loop to end and the loop should, in fact, end when the user enters -1. Save all code for this problem in a file like `largest_number.py`. Don't forget to include comments for your code and to organize your code in a reasonable fashion.
 - b. The main body of the program (should follow the function definition) should call the function for testing purposes.

Hint: it will probably be easier if you use more than one variable to help you keep track of the largest number entered, but you shouldn't need a lot of variables either.

- In english (not code), what is your algorithm for your function/solution?

.....

.....

.....

.....

.....

.....

.....

.....

- Did the version of your program you are submitting produce any errors (running in IDLE or auto-grader, if applicable)? How might you revise your algorithm and/or program?

.....

.....

.....

.....

.....

- Were you able to solve this task? If not, can you think of anything else to try?

.....

.....

.....

.....

.....

.....

2. Running for Congress: A person is eligible to be a US senator if they are at least 30 years old and have been a US citizen for at least 9 years. To be a US representative, you need to be at least 25 years old and have been a US citizen for at least 7 years. Write a function called `eligible_to_run` that takes two numbers indicating a person's age and years of citizenship, then prints a message stating their eligibility for the Senate and House.

The message should have one of the following forms:

- Congratulations! You are eligible to run for the House of Representatives and the Senate.
- You are eligible to run for the House of Representatives, but not for the Senate.
- Sorry! You are not eligible to run for Congress.
- In english (not code), what is your algorithm for your function/solution?

.....

.....

.....

.....

.....

.....

- Make sure to test your implementation thoroughly. What are the different scenarios that should be tested? Show what you test by including a list of all function calls at the bottom of your file.
- (Optional 1) In addition to the citizenship and age requirements, an individual must be a resident of the state they are seeking to represent. Adjust the function you wrote to consider whether the person is a resident of the state or not.
- (Optional 2) In order to be President of the US, you need to be at least 35 years old, a natural born citizen, and have lived in the US for at least 14 years. Create a new function to check whether the person is eligible to become US president.
- Did the version of your program you are submitting produce any errors (running in IDLE or autograder, if applicable)? How might you revise your algorithm and/or program?

.....

.....

.....

- Were you able to solve this task? If not, can you think of anything else to try?

.....

.....

.....

.....

For portfolio submission, insert you printed program pages here.

A3) A bit of review

1. The code below should print "One" if the variable a equals 1, "Two" if a equals 2, "Three" if a equals 3, and "Other" if a is something else. But it prints "One" and "Other" if a equals 1, "Two" and "Other" if a equals 2, "Three" if a equals 3, and "Other" if a is something else. Explain what is wrong and fix it.

```
if a == 1:
    print("One")
if a == 2:
    print("Two")
if a == 3:
    print("Three")
else:
    print("Other")
```

2. Consider the following problem specification: Define a function that doubles a numeric (int or float) parameter. Which of the following successfully implement this function and demonstrate to a user that it works? For those that don't satisfy both requirements, explain what the problem(s) is/are.

a.

```
def double(a):
    result = 2*a
    return result

print(result)
```

.....
.....
.....
.....
.....

b.

```
def double(a):
    result = 2*a
    return result

double(10)
print(result)
```

.....
.....
.....
.....
.....

c.

```
def double(a):
    result = 2*a
    return result

double(10)
print(return)
```

.....
.....
.....
.....
.....

d. `def double(a):`
 `result = 2*a`
 `return result`

 `x = double(10)`
 `print(x)`

.....
.....
.....
.....

e. `def double(a):`
 `result = 2*a`
 `return result`

 `x = double(10)`
 `print(double(10))`

.....
.....
.....
.....

f. `def double(a):`
 `result = 2*a`
 `return result`

 `print(double(10))`

.....
.....
.....
.....

g. `def double(a):`
 `result = 2*a`
 `return result`

 `x = double(10)`
 `print(double(10))`

.....
.....
.....
.....

h. `def double(a):`
 `result = 2*a`
 `print result`

 `print(double(10))`

.....
.....
.....
.....

i. `def double(a):`
 `result = 2*a`
 `print result`

 `double(10)`

.....
.....
.....
.....

A4) Leap year & Dice

1. Leap years: A year is a leap year if it is divisible by 4, unless it is a century year (divisible by 100) that is not divisible by 400. Some examples:

- 2020 was a leap year because it is divisible by 4.
- 1900 was not a leap year, because it is divisible by 100, but not 400
- 2000 was a leap year, because it is divisible by 400

Write a function that takes a year as a parameter and determines whether or not it is/was a leap year.

- In english (not code), what is your algorithm for your function/solution?

.....

.....

.....

.....

.....

.....

- (Optional) Add a while loop to the main body of the program that:

- Asks the user to enter the year they want to check
- Calls the function you created
- Asks the user if they want to check another year (y/n)

- Did the version of your program you are submitting produce any errors (running in IDLE or auto-grader, if applicable)? How might you revise your algorithm and/or program?

.....

.....

.....

.....

- Were you able to solve this task? If not, can you think of anything else to try?

.....

.....

.....

.....

.....

2. Rolling Dice: Write a function that takes two parameters, faces and num_dice that indicate how many sides a single die has and how many dice it should roll respectively. (`random.randint` may be helpful here.)

b. Add two while loops that check the numbers the user enters:

- The first loop should check the number of faces and should continue running until the user provides an integer larger than one
- The second loop should check the number of dice the user will roll and should continue running until the user enters an integer larger than zero +These loops ensure that we have at least one side on each "die" and that we roll at least one die.

c. Call the function you wrote to roll the dice

d. Add another while loop that encapsulates the previous two loops as well as the call to the function.

- This loop should ask the user if they want to roll another set of dice (y/n) and continue so long as the user indicates they want to continue
- In english (not code), what is your algorithm for your function/solution?

.....
.....
.....
.....
.....
.....

- Did the version of your program you are submitting produce any errors (running in IDLE or autograder, if applicable)? How might you revise your algorithm and/or program?

.....
.....
.....
.....

- Were you able to solve this task? If not, can you think of anything else to try?

.....
.....
.....
.....
.....

A5) Written Questions

1. Which of the following are expressions? Which of the expressions are Boolean expressions (ie. conditions)? What value is described by each of the Boolean expressions? Assume that the following variables have been defined:

`x = 7`

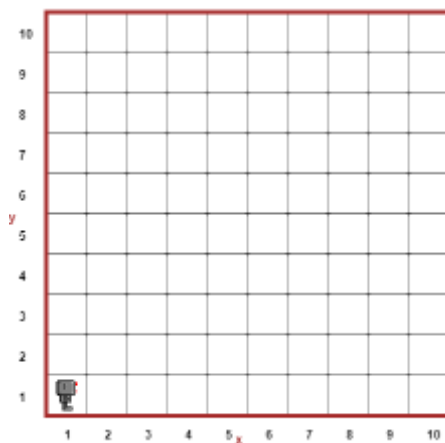
`y = 2`

	expression (y/n)?	Boolean (y/n)?	value
<code>x - 5 == y</code>			
<code>if x < y:</code>			
<code>return x</code>			
<code>x // 3 != 2</code>			
<code>x > 4</code>			
<code>x % 3</code>			
<code>x = 7</code>			
<code>20 = y</code>			

2. Consider the following code snippets. For each of them, what is Reeborg going to do when this code is run? You can assume that Reeborg is carrying an unlimited supply of tokens and is starting in the bottom left corner facing East as usual.

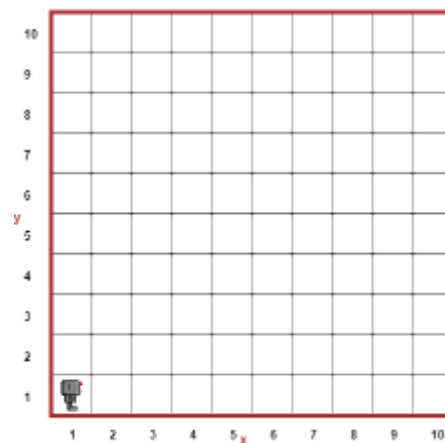
- a. Sketch the behavior of Reeborg for each of the two code snippets

```
while front_is_clear():
    put()
    move()
```



(a) a

```
while front_is_clear():
    put()
    move()
```



(b) b

b. Explain why Reeborg is behaving this way in each scenario.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

A6) Cash register & sum of powers

1. Summing up numbers. Write a function called `cash_register` that repeatedly asks the user to input (positive) numbers and sums them all up. When the user inputs a negative number, the function should print out a message to let the user know their total. The negative number should not be included in the sum.

For example, one run of the function might look like this:

```
>>> cash_register()
Welcome to the cash register!
Please enter the price of each of your items.
Please enter a negative number to indicate when you're finished.

Please enter the price. (Input a negative to stop.) 2
Please enter the price. (Input a negative to stop.) 3.75
Please enter the price. (Input a negative to stop.) 9
Please enter the price. (Input a negative to stop.) 6
Please enter the price. (Input a negative to stop.) -2
Your total is: 20.75
```

- In english (not code), what is your algorithm for your function/solution?

.....

.....

.....

.....

.....

.....

- Did the version of your program you are submitting produce any errors (running in IDLE or auto-grader, if applicable)? How might you revise your algorithm and/or program?

.....

.....

.....

.....

- Were you able to solve this task? If not, can you think of anything else to try?

.....

.....

.....

.....

2. Sums of Powers. Sums of powers arise in many contexts in maths. A sum of powers is a sequence of the first n integers raised to a given power. For example, the sum of the first 5 squares is:
 $1^2 + 2^2 + 3^2 + 4^2 + 5^2 = 55$

- a. Download the starter file `sum_of_powers.py` from the course website.
- b. Write a function called `sum_of_powers` which, given an exponent and an integer n that indicates how many numbers to sum up, calculates (and returns) the first n integers raised to the given exponent.
- c. Save your program in a file called `sum_of_powers.py`.
- d. Select a few inputs for your function and calculate what the correct output of your function should be for those inputs by hand (or with a calculator). Use these inputs to test your function.

- In english (not code), what is your algorithm for your function/solution?

.....

.....

.....

.....

.....

.....

.....

.....

.....

- Did the version of your program you are submitting produce any errors (running in IDLE or auto-grader, if applicable)? How might you revise your algorithm and/or program?

.....

.....

.....

.....

- Were you able to solve this task? If not, can you think of anything else to try?

.....

.....

.....

.....

For portfolio submission, insert you printed program pages here.

A7) Reeborg around the world & mountain climbing

1. Reeborg: Around the world

Use the world "Around - variable" from the "Introduction" collection of worlds. Every time the world is reloaded, it gives you a rectangular world of a different size. Additionally, Reeborg is carrying an infinite number of tokens. Write a program that makes Reeborg walk around the perimeters of the world exactly once, no matter how big the world is.

Hint: is there a way to mark the square(s) that you've been to before?

- In english (not code), what is your algorithm for your solution?

.....

.....

.....

.....

.....

.....

.....

.....

- Did the version of your program you are submitting produce any errors (running in Reeborg or autograder, if applicable)? How might you revise your algorithm and/or program?

.....

.....

.....

.....

.....

.....

.....

- Were you able to solve this task? If not, can you think of anything else to try?

.....

.....

.....

.....

.....

.....

2. Reeborg: mountain climbing. Download the `mounatin_top` folder of starter code from the course website. It should include four different worlds, each of which starts with Reeborg at the foot of a mountain and the goal is to help Reeborg climb to the top (Note: Reeborg should climb not fly). Write a program using function definitions, loops, and conditional statements that help Reeborg climb the mountain in any of the four worlds that are given and in any other similar mountain environments.

- In english (not code), what is your algorithm for your solution?

.....

.....

.....

.....

.....

.....

.....

.....

- Did the version of your program you are submitting produce any errors (running in Reeborg or autograder, if applicable)? How might you revise your algorithm and/or program?

.....

.....

.....

.....

.....

- Were you able to solve this task? If not, can you think of anything else to try?

.....

.....

.....

.....

.....

.....

.....

For portfolio submission, insert you printed program pages here.

A8) Tracing while loops

Please try to solve these problems without running/writing code. Instead you should try to come up with your answers by tracing through the code yourself and reasoning about what the Python interpreter does.

Note: You will see similar questions on the Pen and Paper Midterm.

1. What gets printed by the following code?

```
1) x = 10
2) while x > 8:
3)     print(x, end = " ")
4)     x = x - 1
```

.....

2. What gets printed by the following code?

```
1) x = 10:
2) while x > 8:
3)     x = x - 1
4)     print(x, end = " ")
```

.....

3. What gets printed by the following code?

```
1) i = 0
2) while i < 3:
3)     print(i, end = " ")
```

.....

4. What gets printed by the following code?

```
1) valid = False
2) while not valid:
3)     x = int(input ("Enter a number: "))
4)     valid = (x % 2 == 1 and x % 3 == 0)
```

.....

5. After writing down your answers, check each code chunk in Python. How many were you able to successfully trace before running the code?

.....

6. Prompt: Is a `while` loop a way for a program to observe things? Is the program making an informed decision based on observing its current memory state (or the state of variables)?

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

A9) Primes

A prime number is a natural number greater than 1 that is only evenly divisible by itself and 1. In this assignment you will define a function called `is_prime` that checks whether a given number is a prime number.

1. Create a file called `primes.py`, which you'll use for this assignment.
2. Before writing any code, answer the following questions.
 - a. How many parameters does this function need and what do they represent?
 - b. Should this function have a return value? If so, what kind of value will be returned? (e.g. number, string, boolean?)
 - c. Give a few example function calls that you could use to test whether your implementation works correctly.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

3. The name of the function you create should be `is_prime`. Below is an algorithm you can use for your function. Note: You only have to worry about positive integers.
 - a. Assume that `x` is the number that you're checking to determine if it's prime.
 - b. If `x` is smaller than 2, it is not a prime number, so you can return `False` immediately.
 - c. Otherwise, define a boolean variable, which represents whether or not `x` is prime. Initialize this variable to (i.e. assign it the value) `True`. Note: in effect, we begin by assuming that `x` is prime.
 - d. For all numbers between 2 and `x`, check whether `x` is evenly divisible by the number. If so, `x` is not prime. Record this by changing the value of the boolean variable you defined in the previous step.

- e. At the end, the value of the boolean variable should reflect whether or not x is prime, so all you need to do is return that value.

4. (Optional) Can you improve on this algorithm to make it more efficient?

Hint 1: Aside from x itself, what is the largest number x can possibly be evenly divisible by?

Hint 2: Are there any numbers between 2 and x that you don't need to check?

- Did the version of your program you are submitting produce any errors (running in Reeborg or autograder, if applicable)? How might you revise your algorithm and/or program?

.....

.....

.....

.....

.....

.....

.....

.....

- Were you able to solve this task? If not, can you think of anything else to try?

.....

.....

.....

.....

.....

.....

.....

.....

For portfolio submission, insert you printed program pages here.

A10) Number guessing

1. Your task is to write a program that plays a number guessing game. Download the starter code `number_guessing.py`. The computer chooses a random number between 1 and 100, then the user makes guesses until they find the right number. After every guess, the computer gives a hint by saying whether the number the user guessed was too high or too low.

For example, the interaction could look like this. The numbers after the colon are what the user/player types in.

So: the computer prints out "Guess a number between 1 and 100: "and then the user types in 50.

```
>>> number_guessing()
Guess a number between 1 and 100: 50
That is too low. Try again: 75
That is too high. Try again: 67
That is too high. Try again: 58
That is too high. Try again: 54
That's right! My number was: 54
```

Some details:

- The computer should choose a number between 1 and 100 inclusive (i.e. $[1, 100]$ to use math notation). That is, 1 is the smallest number the computer picks, and 100 is the largest number the computer picks. You may have to choose different start and stop points depending on which function you use from the random module.
- You should define a function called `number_guessing`. When this function is called, one whole round of the number guessing game is played. That is, when this function is called, the computer chooses a random number and then asks the user for a guess. If the guess is too high, the computer says so and lets the user input another guess; similarly if the guess is too low. This is repeated until the user guesses the right number.
- In english (not code), what is your algorithm for your solution?

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

-
-
- Did the version of your program you are submitting produce any errors (running in Reeborg or autograder, if applicable)? How might you revise your algorithm and/or program?

-
-
-
-
-
-
-
-
-
-
- Were you able to solve this task? If not, can you think of anything else to try?

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

For portfolio submission, insert you printed program pages here.

B1: Share-out

0. Did you share-out this week? If not, and if you would like to make up your credit, where in the upcoming schedule would you like to be added?

.....

.....

.....

.....

1. For which activity did you share-out?

.....

.....

.....

.....

2. What was the solution you shared?

.....

.....

.....

.....

.....

.....

.....

3. If any, what changes did you make to your solution in response to questions or discussion?

.....

.....

.....

.....

.....

.....

.....

B2: Share-out (optional)

0. Did you share-out this week? If not, and if you would like to make up your credit, where in the upcoming schedule would you like to be added?

.....

.....

.....

.....

1. For which activity did you share-out?

.....

.....

.....

.....

2. What was the solution you shared?

.....

.....

.....

.....

.....

.....

.....

3. If any, what changes did you make to your solution in response to questions or discussion?

.....

.....

.....

.....

.....

.....

.....